

Document number: P4020R0
Date: 2026-02-23
Audience: EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Concerns about contract assertions

With contract assertions ([\[P2900R14\]](#)) voted into the [\[CD\]](#), a number of concerns have been risen. A number of claims — both in favor and against it — have been made. In this paper we address these concerns and offer some reflections on contracts in general, and contract assertions in [\[P2900R14\]](#).

Contracts in general

Contracts are a thing in any serious programming language. They require no "syntax" or "feature". If one person creates a component and another uses it, in order for this coöperation to work both parties need to have the same understanding on what this component does. This shared understanding is built outside of the programming language: via meetings, discussions, training, and most notably: documentation. A function contract is more than just a set of preconditions and postconditions. Consider:

```
double atan2(double, double);
```

In this function, which of the parameters represents the *x* and which the *y* coordinate?

```
void draw_rect(int, int, int, int);
```

In this function what do the four numbers represent?

- the upper-left and the lower-right corners or
- the origin and the extents?

In either case, the function contract will tell you.

Very importantly, this exist today. Serious C++ programmers have to and do deal with contracts: they recognize them, they communicate them, and they will continue to do so with or without a dedicated language feature. C++ doesn't need a new feature for this to continue.

So why have we been working to add precondition and postcondition declarations to the language for over two decades?

- To provide one more means of formally communicating at least parts of the contract.
- To communicate, at least parts of the contract, also to machines.
- To enable (or guarantee) that parts of these contracts are "enforced", that is, as in sanitizers, an instrumentation code is injected to kill the program that is observed to violate the declared preconditions (and postconditions).
- To have a standardized, uniform mechanism of recording any act of contract violation in the program.

P2900 contract assertions

Getting the necessary confidence

Some proponents of P2900 claim they have experience with "a thing like" P2900. This is why they are confident it will work. Similarly, the proponents of [\[P3911R2\]](#) claim that they have experience with working with a pair of tools — a conditional and an unconditional enforcement — and therefore they are confident that adding a similar pattern on top of P2900 will work.

Both these claims, in the author's opinion, are unfounded. The declared experience is with using *statements* inside function bodies. There, it is the function's implementation detail whether a check is performed or not. Of course, this checking can be documented, enforced and tested, but still the implementation, the deployment and the usage experience is about dealing with statements in function bodies: not annotations in function declarations.

We have seen reports that ever since P2900 has been implemented as an experiment in GCC and Clang, experiments have been performed with rewriting some libraries using assertion statements into a variant using precondition and postcondition assertions. This qualifies as "some implementation experience" and "some deployment experience". But in the case of contracts this is not enough to address the reported concerns. For this feature, the reported concerns are unique. They concern things like

- How many vulnerabilities will not be prevented despite the expectations (founded or not).
- What is really guaranteed for the users and what is not.
- How many new vulnerabilities will surface.
- How many promises regarding the configurability can be fulfilled in different toolchains.
- How many new paths in the programs will be created, how many new bugs added.
- What the runtime cost is when the programmers or library authors want the contract assertions enforced, in different implementations, environments, toolchains.
- How the feature interacts with the *long* process of software development life-cycle.

Getting the necessary confidence over the above questions requires more evidence on people and companies being able to successfully use the feature, across a longer time span, before the Committee can commit to standardizing the feature.

There is a lot of experience with configurable assertions inside function bodies. There is little experience with assertions in function declarations.

Objections from vendors

The representatives of two compiler vendors — Microsoft and EDG — have objected to standardizing contract assertions as in P2900. The objections are not about implementability. The feature is fairly simple to implement in its minimal form (just type-check the conditions and otherwise ignore them). They are about the (un)usefulness and causing harm to their users. It is admittedly surprising that this fact alone does not automatically disqualify the feature in its present form from standardization.

Adding last-minute patches

During the National Ballot phase, EWG has reviewed a number of papers in response to the National Body comment [RO 2-056](#). Things proposed in [\[P3911R2\]](#) and other considered papers (P4005R0, P4009R0) are new features, features proposed on top of P2900 for C++26.

This cannot work. EWG must make a decision whether contract assertions in the CD are defective or not. If they are deemed defective and a simple fix is impossible (and it isn't: it is the design and the phased approach that are being questioned) they should be removed. Otherwise any new feature requests should automatically be rejected at this stage.

If a feature is deemed defective before it has been standardized, standardizing competing alternatives alongside seems like the wrong thing to do.

The idea behind P2900 is difficult to adopt

The authors of P2900 have repeatedly communicated the idea, the model and the motivation behind the proposal. It is to *declare* what is a precondition and what is a postcondition of a given function. It is not to *enforce* or guarantee any behavior, at least not from the level of the language.

It is evident that this model is not adopted by a significant portion of WG21. This is an important data point in itself. The model may be right, sound, coherent and usable, but if the community for whatever reason cannot accept it, it will fade.

And admittedly, the implementation of the model has its flaws. People already discover how it can be abused to achieve the effect of arbitrary function prologues and epilogues.

Statements in function declarations

A number of proposals reviewed by EWG in response to [RO 2-056](#) effectively propose to add statements to function declarations, or in other words, introduce a form of prologues and epilogues to functions. While this may appear "expressive", from a maintainability perspective it makes code more difficult to reason about: some code gets executed but we cannot see it in the source code. Even the biggest blessing of C++ — destructors — comes at a cost: "why is some code executed when I see no statement in scope?"

Recommendations

There seem to be only two coherent responses to [RO 2-056](#):

1. Take no action (ship as is).
2. Implement solution "v4" from [\[P3911R0\]](#) (move C++ contracts to other shipping vehicle).

References

- [CD] — Jonathan Caves, Daniel Kruegler, Nina Ranns, Tim Song, "ISO/IEC CD 14882, Programming languages — C++",
(<https://www.iso.org/standard/91179.html>).
- [N4075] — John Lakos, Alexei Zakharov, Alexander Beels, "Centralized Defensive-Programming Support for Narrow Contracts (Revision 6)",
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4075.pdf>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++",
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2900R14] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++",
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2900r14.pdf>).
- [P3376R0] — Andrzej Krzemiński, "Contract assertions versus static analysis and 'safety'",
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3376r0.html>).
- [P3829R0] — Chisnall, Spicer, Voutilainen, Dos Reis, Garcia, "Contracts do not belong in the language",
(<https://isocpp.org/files/papers/P3829R0.pdf>).
- [P3835R0] — John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, "Contracts make C++ less safe -- full stop!",

(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3835r0.html>).

- [P3911R0] — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3911r0.html>).
- [P3911R2] — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, Herb Sutter, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3911r2.html>).